DZone

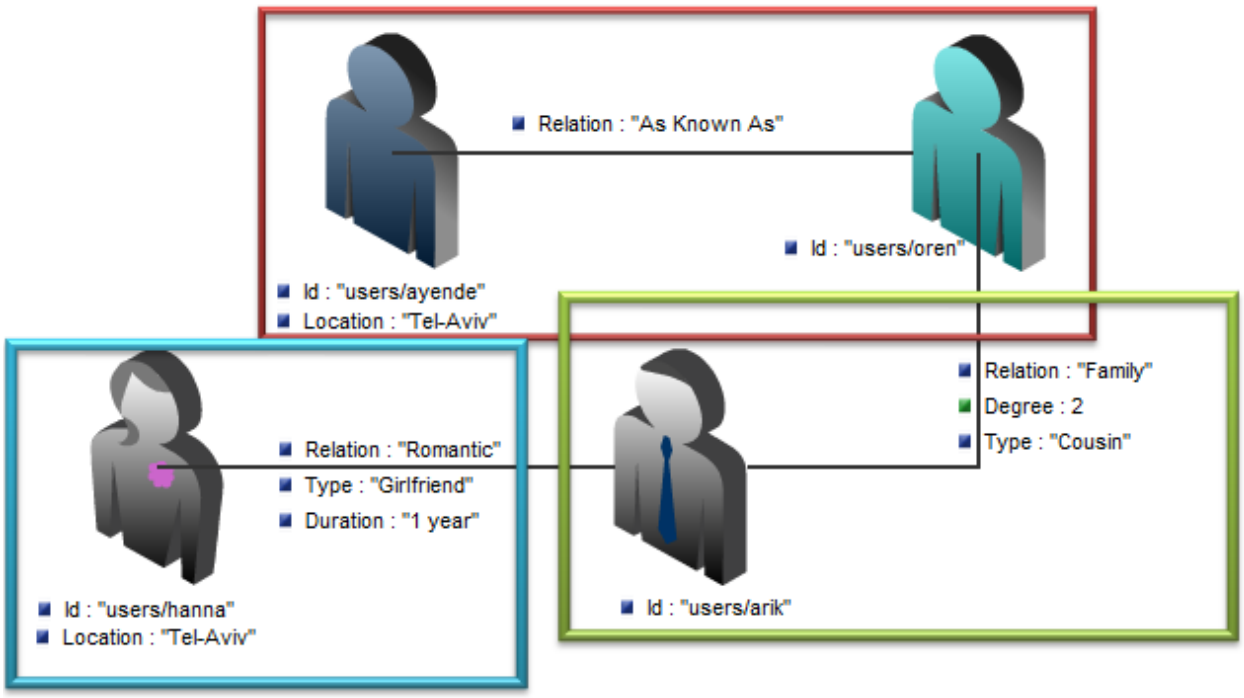
On Scaling Graph Databases

by Oren Eini, CEO RavenDB MVB · Jul. 18, 12 · Database Zone · Not set

RavenDB vs MongoDB: Which is Better? This White Paper compares the two leading NoSQL Document Databases on 9 features to find out which is the best solution for your next project.

Yesterday I talked about graph databases, outlining what they are and how they work. One of the interesting things about this series is that in many cases, I am posing a question (to myself), trying to answer it, then go and find out what other people do.

When thinking about scaling scenarios for a graph database, I had the following scenario in mind, a graph of nodes that is spread across multiple servers, where each member in the graph may reside on any machine in the system. The following diagram demonstrate what I am thinking about, each rectangle represent a different machine in the system:



Why is this important?

A single machine solution is obviously a barrier to scaling (and safety, but that is another concern. In a graph database, having relations between the node is *the point*, that makes sharding a bit more complicated, because unless you store the entire graph on a single machine, you are forced to query across machine boundaries. And you *can't* store a graph in a single machine, for the simple reason that it is unlikely that you can limit a graph to be that small. Think about the implications of Six Degrees of Separation for graph databases and it will be clear what the problem is. In real world graphs, *everyone* is connected to everyone.

The problem with breaking the entire graph across machines is that now it is much more expensive to do graph traversals. The following query, which previous run on a single machine:

```
new GraphDatabaseQuery
{
   SourceNode = ayende,
   MaxDepth = 3,
   RelationsToFollow = new[]{"As Known As", "Family", "Friend", "Romantic", "Ex"},
   Where = node => node.Location == ayende.Location,
   SearchOrder = SearchOrder.BreadthFirst
}.Execute();
```

Now need to touch 3 different machines. Worse, it isn't the number of machines that impacts that, but the spread of graph nodes across machines in the system.

After spending some time thinking about it, I came to the conclusion that I can't envision any general way to solve the problem. Oh, I can think of several ways of reduce the problem:

- Batching cross machine queries so we only perform them at the close of each breadth first step.
- Storing multiple levels of associations (So "users/ayende" would store its relations but also "users/ayende"'s relation and "users/arik"'s relations).

The solution most likely to be successful is limiting the depth of cross machine node searches. In many cases, that is acceptable, I think. If we put the depth limit on 3, we can still give pretty good answers in a reasonable time frame. But the only way this can be made to work is with good batching support.

The algorithm may look like:

```
public IEnumerable<Node> DistributedTraverse(Node sourceNode, int depth, string relationToFollow, Func<Node, filter> predicate)
{
    if(depth == 0) // feeling defensive
        yield break;

    var related = GetRelatedNodes(sourceNode.ShardName, relationToFollow, predicate);

    foreach(var result in related)
            yield return result;

    if(depth == 1) // don't even bother asking down the line
    {
        yield break;
    }

    foreach(var relationsByShard in related.GroupBy(x=>x.ShardName))
    {
        var shard = GetShardProxy(relationsByShard.Key);
        var results = shard.BatchedQuery(sourceNodes: relationsByShard.ToArray(), depth - 1,relationToFollow, predicate);
        foreach(var result in results)
            yield return result;
    }
}
```

This give us a maximum amount of *(depth * number_of_machines_in_cluster) – depth* remote calls: With a depth of 3 and 3 machines in the cluster, we would have a max of 6 calls.

With that theory out of our heads, let us examine how real world Graph DBs tried to resolve this issue…

Neo4j (which seems to be pretty much the default for Graph DBs) doesn't handle this currently, there are some hints that they intend to offer cluster wide replication, but nothing about design or implementation details. Neo4j does offer write-master/read-slaves approach for scaling out, which is really nice, but even that approach is limited at one point, and in this post, I am focusing on what happen when you go beyond that point.

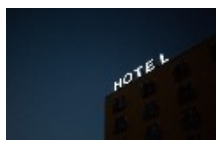
FlockDB (which is what is used by twitter) does include, as part of its design goals: "horizontal scaling including replication". However, FlockDB isn't trying to solve the problem outlined above, indeed, graph traversal is a *non goal* for it. FlockDB is more about finding one level of relations *very* fast than anything else.

In summary, I believe that while you can shard a graph database, it place a very lot limit on the type of graph walking queries you can make. Now, just to give you an idea, Neo4j, for example, appears to be able to routinely handle billions on nodes on a single machines, so you might no need to scale higher than that..

Get comfortable using NoSQL in a free, self-directed learning course provided by RavenDB. Learn to create fully-functional real-world programs on NoSQL Databases. Register today.

Like This Article? Read More From DZone

Customize HTTP Error Responses in Spring Boot



Marriott Confirms Breach Impacts as Many as 500 Million Guests

Node.js vs. Django: Is JavaScript Better Than Python?

Free DZone Refcard
Temporal Data Processing

Topics:

Published at DZone with permission of Oren Eini, CEO RavenDB , DZone MVB. See the original article here.
Opinions expressed by DZone contributors are their own.

Database Partner Resources

:oros NoSQL Performance Benchmark 2018: Compare Couchbase Server, MongoDB, and DataStax Enterprise
ouchbase